

TrustLot System - UML Diagrams

1. UML Class Diagram - Complete System

```
classDiagram
class AppUser {
    -int id
    -string username
    -string name
    -string phoneNumber
    -string email
    -string passwordHash
    -Role role
    -timestamp createdAt
    -timestamp updatedAt
    +register(username, name, phone, email, password, role)
    +login(username, password)
    +logout()
    +updateProfile(name, phone, email)
}

class Car {
    -int id
    -int sellerId
    -string make
    -string model
    -int year
    -decimal price
    -int mileage
    -string color
    -boolean available
    -ApprovalStatus approvalStatus
    -int viewCount
    -CarCondition condition
    -FuelType fuelType
    -TransmissionType transmission
    -string bodyType
    -int numberOfOwners
    -int engineCc
    -boolean insured
    -boolean pucValid
    -string description
    -timestamp createdAt
    +addCar(details)
    +updateCar(details)
    +deleteCar()
    +getCarDetails()
    +incrementViewCount()
}

class PurchaseOrder {
    -int id
    -int buyerId
    -int sellerId
```

```

    -int carId
    -int paymentId
    -OrderStatus status
    -boolean fraudAlert
    -timestamp createdAt
    +initiatePurchase(car, paymentInfo)
    +processPayment()
    +approveBuyer()
    +rejectBuyer()
    +approveSeller()
    +rejectSeller()
    +getOrderStatus()
    +getReceipt()
}

class Payment {
    -string id
    -PaymentStatus status
    -decimal amount
    -string method
    -string gatewayName
    -string gatewayTransactionId
    -string failureReason
    -timestamp paidAt
    +processPayment(amount, method, token)
    +handlePaymentSuccess()
    +handlePaymentFailure(reason)
    +refundPayment()
}

class SupportTicket {
    -int id
    -int userId
    -string subject
    -string description
    -TicketStatus status
    -timestamp createdAt
    -timestamp updatedAt
    -List~TicketResponse~ responses
    +createTicket(subject, description)
    +addMessage(message)
    +closeTicket(response)
    +reopenTicket()
    +getMessages()
}

class TicketResponse {
    -int id
    -int ticketId
    -int senderId
    -string message
    -timestamp respondedAt
    +addResponse(message)
    +editResponse(newMessage)
}

```

```

        +deleteResponse()
    }

class Feedback {
    -int id
    -int userId
    -string message
    -timestamp createdAt
    +submitFeedback(message)
    +viewFeedback()
}

class SellerReview {
    -int id
    -int reviewerId
    -int sellerId
    -int rating
    -string comment
    -timestamp createdAt
    +submitReview(rating, comment)
    +updateReview(rating, comment)
    +deleteReview()
    +getAverageRating()
}

class WishListItem {
    -int id
    -int buyerId
    -int carId
    -timestamp addedAt
    +addToWishlist(car)
    +removeFromWishlist(car)
    +viewWishlist()
    +isInWishlist(car)
}

class RecentView {
    -int id
    -int userId
    -int carId
    -timestamp viewedAt
    +recordView(car)
    +getRecentViews()
    +clearRecentViews()
}

class AuthService {
    -SessionTokenService tokenService
    +registerUser(userDetails)
    +loginUser(username, password)
    +logoutUser(token)
    +validateToken(token)
    +refreshToken(token)
    +changePassword(userId, oldPassword, newPassword)
}

```

```

}

class CarService {
    -CarRepository carRepository
    +listCar(carDetails)
    +updateCar(carId, carDetails)
    +deleteCar(carId)
    +getAvailableCars()
    +getCarDetails(carId)
    +searchCars(filters)
    +compareCars(carId1, carId2)
    +getPopularCars()
}

class PurchaseService {
    -PaymentService paymentService
    -CarService carService
    -ApprovalService approvalService
    +initiatePurchase(buyerId, carId, paymentInfo)
    +processPurchaseFlow(orderId)
    +getMyOrders(userId)
    +getPendingSales(sellerId)
    +downloadReceipt(orderId)
}

class PaymentService {
    -PaymentGateway gateway
    -PaymentRepository paymentRepository
    +processPayment(amount, method, token)
    +handleGatewayResponse(response)
    +recordPayment(paymentDetails)
    +refundPayment(paymentId)
}

class AdminService {
    -CarRepository carRepository
    -PurchaseOrderRepository orderRepository
    -UserRepository userRepository
    +getPendingCars()
    +approveCar(carId)
    +rejectCar(carId, reason)
    +getPendingOrders()
    +approveOrder(orderId)
    +rejectOrder(orderId)
    +editUser(userId, details)
    +getDashboardStats()
    +getGraphData()
}

class WishlistService {
    -WishlistRepository wishlistRepository
    +addToWishlist(userId, carId)
    +removeFromWishlist(userId, carId)
    +getWishlist(userId)
}

```

```

        +isWishlisted(userId, carId)
    }

    class ReviewService {
        -ReviewRepository reviewRepository
        +submitReview(reviewId, sellerId, rating, comment)
        +updateReview(reviewId, rating, comment)
        +getSellerReviews(sellerId)
        +getAverageRating(sellerId)
        +isEligibleToReview(buyerId, sellerId)
    }

    class SupportTicketService {
        -SupportTicketRepository ticketRepository
        +createTicket(userId, subject, description)
        +addMessage(ticketId, userId, message)
        +closeTicket(ticketId, response)
        +getMyTickets(userId)
        +getAllTickets()
    }

    class ApprovalService {
        +checkFraudRisk(order)
        +validatePurchase(car, buyer, seller)
        +approvePurchase(orderId)
        +rejectPurchase(orderId)
    }

    class SessionTokenService {
        -Map<String, SessionToken> activeSessions
        +generateToken(userId)
        +validateToken(token)
        +invalidateToken(token)
        +extendTokenExpiry(token)
        +cleanupExpiredSessions()
    }

    class PaymentGateway {
        +processPayment(amount, method, token)
        +refund(paymentId)
        +checkPaymentStatus(paymentId)
    }

    class AuthController {
        -AuthService authService
        +register(registerRequest)
        +login(loginRequest)
        +logout(token)
    }

    class CarController {
        -CarService carService
        +addCar(carRequest)
        +updateCar(carId, carRequest)
    }

```

```

    +deleteCar(carId)
    +getAvailableCars()
    +getCarDetails(carId)
    +compareCars(carId1, carId2)
    +getPopularCars()
}

class PurchaseController {
    -PurchaseService purchaseService
    +purchaseCar(carId, paymentInfo)
    +getMyOrders()
    +getPendingSales()
    +approveSale(orderId)
    +rejectSale(orderId)
    +getReceipt(orderId)
}

class AdminController {
    -AdminService adminService
    +getPendingCars()
    +approveCar(carId)
    +rejectCar(carId)
    +getPendingOrders()
    +approveOrder(orderId)
    +rejectOrder(orderId)
    +getUsers()
    +editUser(userId, details)
    +getDashboard()
    +getGraphData()
}

AppUser "1" -- "*" Car : lists
AppUser "1" -- "*" PurchaseOrder : buys
AppUser "1" -- "*" PurchaseOrder : sells
AppUser "1" -- "*" SupportTicket : creates
AppUser "1" -- "*" SellerReview : gives
AppUser "1" -- "*" Feedback : submits
AppUser "1" -- "*" WishlistItem : adds
AppUser "1" -- "*" RecentView : views
AppUser "1" -- "*" TicketResponse : sends

Car "1" -- "*" PurchaseOrder : involves
Car "1" -- "*" WishlistItem : in
Car "1" -- "*" RecentView : viewed_in

PurchaseOrder "1" -- "1" Payment : has
PurchaseOrder "1" -- "*" SupportTicket : related_to

SupportTicket "1" -- "*" TicketResponse : contains

AuthService --> SessionTokenService
CarService --> Car
PurchaseService --> Payment
PurchaseService --> Car

```

```

PurchaseService --> ApprovalService
PaymentService --> PaymentGateway
PaymentService --> Payment
AdminService --> Car
AdminService --> PurchaseOrder
AdminService --> AppUser
WishlistService --> WishlistItem
ReviewService --> SellerReview
SupportTicketService --> SupportTicket
SupportTicketService --> TicketResponse

AuthController --> AuthService
CarController --> CarService
PurchaseController --> PurchaseService
AdminController --> AdminService

```

2. UML Sequence Diagram - Car Purchase Flow

```
sequenceDiagram
```

```

actor Buyer
participant Frontend
participant PurchaseController
participant PurchaseService
participant PaymentService
participant PaymentGateway
participant OrderRepo
participant CarService
participant Database

```

```

Buyer->>Frontend: Click "Purchase Car"
Frontend->>Buyer: Show payment form
Buyer->>Frontend: Enter payment details
Frontend->>PurchaseController: POST /purchase

```

```
PurchaseController->>PurchaseService: initiatePurchase(buyerId, carId, paymentInfo)
```

```

PurchaseService->>CarService: validateCar(carId)
CarService->>Database: SELECT car WHERE id=carId
Database-->>CarService: Car details
CarService-->>PurchaseService: Validation success

```

```

PurchaseService->>PaymentService: processPayment(amount, method, token)
PaymentService->>PaymentGateway: POST process_payment
PaymentGateway-->>PaymentService: {status: SUCCESS, txnId}
PaymentService->>Database: INSERT into Payment table
Database-->>PaymentService: Payment recorded
PaymentService-->>PurchaseService: Payment success

```

```

PurchaseService->>CarService: setUnavailable(carId)
CarService->>Database: UPDATE car SET available=false
Database-->>CarService: Updated

```

```
PurchaseService->>OrderRepo: createOrder(buyerId, sellerId, carId, paymentId)
```

```

OrderRepo->>Database: INSERT into PurchaseOrder table
Database-->>OrderRepo: Order created with PENDING_ADMIN_APPROVAL status
OrderRepo-->>PurchaseService: Order details

PurchaseService-->>PurchaseController: Order created successfully
PurchaseController-->>Frontend: {status: 201, orderId, message}

Frontend->>Buyer: Show success message
Buyer->>Frontend: View order details
Frontend->>PurchaseController: GET /orders/my
PurchaseController->>Frontend: Return orders

```

3. UML Sequence Diagram - Admin Approval Flow

```

sequenceDiagram
    actor Admin
    actor Seller
    participant AdminFrontend
    participant AdminController
    participant AdminService
    participant OrderRepo
    participant NotificationService
    participant EmailService
    participant Database

    Admin->>AdminFrontend: Login to Dashboard
    AdminFrontend->>AdminController: GET /admin/orders/pending

    AdminController->>AdminService: getPendingOrders()
    AdminService->>OrderRepo: findByStatus(PENDING_ADMIN_APPROVAL)
    OrderRepo->>Database: SELECT orders WHERE status=PENDING_ADMIN_APPROVAL
    Database-->>OrderRepo: Pending orders list
    OrderRepo-->>AdminService: Orders returned
    AdminService-->>AdminController: Pending orders
    AdminController-->>AdminFrontend: Display orders

    Admin->>AdminFrontend: Review order details
    Admin->>AdminFrontend: Click "Approve Order"
    AdminFrontend->>AdminController: POST /admin/orders/{id}/approve

    AdminController->>AdminService: approveOrder(orderId)
    AdminService->>OrderRepo: updateOrderStatus(orderId, PENDING_SELLER_APPROVAL)
    OrderRepo->>Database: UPDATE PurchaseOrder SET status=PENDING_SELLER_APPROVAL
    Database-->>OrderRepo: Updated
    OrderRepo-->>AdminService: Success

    AdminService->>NotificationService: notifySeller(sellerId, orderId)
    NotificationService->>EmailService: sendEmail(sellerEmail, message)
    EmailService-->>NotificationService: Email sent

    AdminService-->>AdminController: Approval complete
    AdminController-->>AdminFrontend: Success response
    AdminFrontend->>Admin: Show success message

```


EmailService-->>Seller: Receive email notification
Seller->>AdminFrontend: Login as seller
AdminFrontend->>AdminController: GET /user/sales/pending
AdminController->>AdminFrontend: Pending sales for approval
AdminFrontend->>Seller: Show pending orders

4. UML State Diagram - Purchase Order States

stateDiagram-v2

[*] --> Created

Created --> PaymentProcessing: Payment initiated

PaymentProcessing --> PaymentFailed: Payment fails

PaymentProcessing --> PendingAdminApproval: Payment success

PaymentFailed --> [*]: Transaction cancelled

PendingAdminApproval --> AdminRejected: Admin rejects

PendingAdminApproval --> PendingSellerApproval: Admin approves

AdminRejected --> [*]: Order cancelled

PendingSellerApproval --> SellerRejected: Seller rejects

PendingSellerApproval --> Approved: Seller approves

SellerRejected --> [*]: Order cancelled

Approved --> [*]: Order completed

note right of Created

Initial state when order is created

end note

note right of PaymentProcessing

Payment gateway processing

end note

note right of PendingAdminApproval

Waiting for admin verification

end note

note right of PendingSellerApproval

Waiting for seller confirmation

end note

note right of Approved

Purchase finalized, receipt generated

end note

5. UML Use Case Diagram - Buyer Features

```
usecase UC1 as "Browse Cars"
usecase UC2 as "Search/Filter Cars"
usecase UC3 as "Compare Cars"
usecase UC4 as "View Car Details"
usecase UC5 as "Add to Wishlist"
usecase UC6 as "Remove from Wishlist"
usecase UC7 as "View Recent Searches"
usecase UC8 as "Purchase Car"
usecase UC9 as "Make Payment"
usecase UC10 as "View My Orders"
usecase UC11 as "Download Receipt"
usecase UC12 as "Rate Seller"
usecase UC13 as "Submit Review"
usecase UC14 as "Create Support Ticket"
usecase UC15 as "Message Support"
usecase UC16 as "View Support History"
usecase UC17 as "Submit Feedback"
```

```
actor Buyer as "Buyer/User"
```

```
Buyer --> UC1
Buyer --> UC2
Buyer --> UC3
Buyer --> UC4
Buyer --> UC5
Buyer --> UC6
Buyer --> UC7
Buyer --> UC8
Buyer --> UC9
Buyer --> UC10
Buyer --> UC11
Buyer --> UC12
Buyer --> UC13
Buyer --> UC14
Buyer --> UC15
Buyer --> UC16
Buyer --> UC17
```

```
UC8 .> UC9 : includes
UC8 .> UC11 : includes
UC12 .> UC13 : includes
UC14 .> UC15 : includes
```

6. UML Use Case Diagram - Seller Features

```
usecase UC1 as "List Car for Sale"
usecase UC2 as "Edit Car Listing"
usecase UC3 as "Delete Car Listing"
usecase UC4 as "View My Listings"
usecase UC5 as "View Listing Status"
usecase UC6 as "Wait for Admin Approval"
```

```
usecase UC7 as "Respond to Purchase"
usecase UC8 as "Approve Sale"
usecase UC9 as "Reject Sale"
usecase UC10 as "View Sales History"
usecase UC11 as "Download Receipt"
usecase UC12 as "View Seller Rating"
usecase UC13 as "Create Support Ticket"
usecase UC14 as "Message Support"
```

```
actor Seller as "Seller/User"
```

```
Seller --> UC1
Seller --> UC2
Seller --> UC3
Seller --> UC4
Seller --> UC5
Seller --> UC6
Seller --> UC7
Seller --> UC8
Seller --> UC9
Seller --> UC10
Seller --> UC11
Seller --> UC12
Seller --> UC13
Seller --> UC14
```

```
UC1 .> UC6 : includes
UC7 .> UC8 : includes
UC7 .> UC9 : includes
UC8 .> UC11 : includes
```

7. UML Use Case Diagram - Admin Features

```
usecase UC1 as "View Dashboard"
usecase UC2 as "View Analytics"
usecase UC3 as "View Pending Cars"
usecase UC4 as "Approve Car Listing"
usecase UC5 as "Reject Car Listing"
usecase UC6 as "View All Orders"
usecase UC7 as "View Pending Orders"
usecase UC8 as "Approve Order"
usecase UC9 as "Reject Order"
usecase UC10 as "View All Users"
usecase UC11 as "Edit User Details"
usecase UC12 as "Reset User Password"
usecase UC13 as "View Support Tickets"
usecase UC14 as "Respond to Ticket"
usecase UC15 as "Close Ticket"
usecase UC16 as "View Feedback"
usecase UC17 as "Download Receipt"
```

```
actor Admin as "Admin/Moderator"
```

```
Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4
Admin --> UC5
Admin --> UC6
Admin --> UC7
Admin --> UC8
Admin --> UC9
Admin --> UC10
Admin --> UC11
Admin --> UC12
Admin --> UC13
Admin --> UC14
Admin --> UC15
Admin --> UC16
Admin --> UC17
```

```
UC1 .> UC2 : includes
UC3 .> UC4 : includes
UC3 .> UC5 : includes
UC7 .> UC8 : includes
UC7 .> UC9 : includes
UC13 .> UC14 : includes
UC13 .> UC15 : includes
```

8. UML Activity Diagram - Car Listing Process

```
activity
    start
    :User fills car listing form;
    :Select make, model, year, price;
    :Add condition, fuel type, transmission;
    :Enter mileage, number of owners;
    :Write description;
    if (All fields valid?) then
        :Submit listing;
        :System validates data;
        if (Validation passed?) then
            :Store in database;
            :Set status to PENDING_ADMIN_APPROVAL;
            :Notify user of submission;
            :Send email confirmation;
        else
            :Show validation errors;
            :User corrects data;
            note right : Return to form
        endif
    endif
    else
        :Show validation errors;
        :User corrects data;
        note right : Return to form
    endif
endif
```

```

:Admin reviews listing;
if (Listing acceptable?) then
    :Admin approves;
    :Update status to APPROVED;
    :Set available to true;
    :Car appears in search;
    :Notify seller;
else
    :Admin rejects;
    :Update status to REJECTED;
    :Send rejection reason;
    :Notify seller;
endif

stop

```

9. UML Activity Diagram - Purchase Order Processing

```

activity
    start
    :Buyer selects car;
    :Clicks Purchase button;
    :Reviews car details & price;
    :Enters payment information;
    :Submits purchase;

    :System validates purchase eligibility;
    if (Can purchase?) then
        :Check car available;
        if (Car available?) then
            :Process payment;
            if (Payment successful?) then
                :Mark car unavailable;
                :Create purchase order;
                :Set status PENDING_ADMIN_APPROVAL;
                :Notify buyer & seller;
            else
                :Show payment error;
                :Refund any partial charges;
                :End transaction;
            endif
        else
            :Show car unavailable;
            :End transaction;
        endif
    else
        :Show eligibility error;
        :End transaction;
    endif

    :Admin reviews order;
    if (Admin approves?) then

```

```

:Update status PENDING_SELLER_APPROVAL;
:Notify seller;

:Seller reviews order;
if (Seller approves?) then
    :Update status APPROVED;
    :Mark car as sold;
    :Generate receipt;
    :Send confirmation emails;
else
    :Seller rejects;
    :Update status REJECTED;
    :Refund buyer;
endif
else
    :Admin rejects;
    :Update status REJECTED;
    :Refund buyer;
endif
stop

```

10. UML Deployment Diagram

```

graph TB
    subgraph "Client Layer"
        WebBrowser[" Web Browser<br/>Frontend Application"]
    end

    subgraph "Load Balancer"
        LB[" Load Balancer<br/>nginx/HAProxy"]
    end

    subgraph "Application Servers"
        AppServer1[" App Server 1<br/>Spring Boot<br/>Port 8080"]
        AppServer2[" App Server 2<br/>Spring Boot<br/>Port 8080"]
        AppServer3[" App Server 3<br/>Spring Boot<br/>Port 8080"]
    end

    subgraph "Cache Layer"
        Redis[" Redis Cache<br/>Session Storage"]
    end

    subgraph "Database Layer"
        PrimaryDB[" Primary DB<br/>MySQL 8.0<br/>Write"]
        ReplicaDB[" Replica DB<br/>MySQL 8.0<br/>Read-only"]
    end

    subgraph "External Services"
        PaymentGW[" Payment<br/>Gateway"]
        EmailService[" Email<br/>Service"]
        LogService[" Log<br/>Aggregation"]
    end

```

```

subgraph "Storage"
    FileStorage[" Cloud Storage<br/>S3/GCS<br/>Car Images"]
end

WebBrowser -->|HTTP/HTTPS| LB
LB -->|Routes| AppServer1
LB -->|Routes| AppServer2
LB -->|Routes| AppServer3

AppServer1 -->|Session Data| Redis
AppServer2 -->|Session Data| Redis
AppServer3 -->|Session Data| Redis

AppServer1 -->|Read/Write| PrimaryDB
AppServer2 -->|Read/Write| PrimaryDB
AppServer3 -->|Read/Write| PrimaryDB

AppServer1 -->|Read Only| ReplicaDB
AppServer2 -->|Read Only| ReplicaDB
AppServer3 -->|Read Only| ReplicaDB

PrimaryDB -->|Replication| ReplicaDB

AppServer1 -->|API Call| PaymentGW
AppServer2 -->|API Call| PaymentGW
AppServer3 -->|API Call| PaymentGW

AppServer1 -->|SMTP| EmailService
AppServer2 -->|SMTP| EmailService
AppServer3 -->|SMTP| EmailService

AppServer1 -->|Log Stream| LogService
AppServer2 -->|Log Stream| LogService
AppServer3 -->|Log Stream| LogService

AppServer1 -->|Upload/Download| FileStorage
AppServer2 -->|Upload/Download| FileStorage
AppServer3 -->|Upload/Download| FileStorage

```

Diagram Summary

Diagram	Purpose	Key Focus
Class Diagram	System design	Entities, services, controllers, relationships
Sequence: Purchase	Purchase flow	Interactions between components in purchase
Sequence: Approval	Approval flow	Admin and seller approval interactions
State Diagram	Order lifecycle	State transitions during purchase
Use Case: Buyer	Buyer capabilities	All buyer features and interactions
Use Case: Seller	Seller capabilities	All seller features and interactions

Diagram	Purpose	Key Focus
Use Case: Admin	Admin capabilities	All admin features and interactions
Activity: Listing	Car listing process	Steps in listing a car
Activity: Purchase	Purchase process	Steps in purchasing a car
Deployment	System architecture	Server, database, cache, external services

How to Use These Diagrams

1. **Class Diagram:** Use for understanding system structure and object relationships
2. **Sequence Diagrams:** Use for understanding message flow and timing of operations
3. **State Diagrams:** Use for understanding lifecycle of business entities
4. **Use Case Diagrams:** Use for identifying user interactions and system boundaries
5. **Activity Diagrams:** Use for process documentation and workflow understanding
6. **Deployment Diagram:** Use for understanding production infrastructure and deployment architecture

All diagrams are rendered using Mermaid syntax and are compatible with GitHub, GitLab, and other Markdown renderers that support Mermaid.